Project Management Portfolio: Closed-Loop Dynamic QoS for 5G Core

Arda Özgür · BSc Computer Engineering, Mudanya University
Developed on the IEEE 5G/6G Innovation Testbed

1. Executive Summary

This project is a Proof of Concept for **closed-loop automation** in 5G/6G networks, inspired by the RAN Intelligent Controller (RIC) concept in O-RAN. The system simulates the Physical Resource Block (PRB) utilization of a gNodeB, detects congestion against a configurable threshold (80%), and automatically signals a 5G Core Policy Control Function (PCF) over a 3GPP-style Npcf_SMPolicyControl interface to throttle low-priority subscribers. When load returns below the threshold, the normal profile is restored.

The deliverable is a fully containerised, three-service system (physical-layer simulator, decision engine, mock PCF) with externalised policy configuration and an automated benchmark/reporting pipeline.

Item	Detail
Role	Project owner: architecture, development, integration, testing, documentation
Domain	5G NR, network slicing, QoS, O-RAN closed-loop automation
Duration	7 October 2026 – 19 October 2026
Supervisor / testbed contact	Dr. Öğr. Üyesi Nergis Erdem
Tech stack	Python 3.9, TensorFlow 2.15, NVIDIA Sionna, Flask, PyYAML, Requests, Pandas, Matplotlib, Docker Compose

2. Project Charter

2.1 Problem Statement

Traditional cellular QoS policies are static, fixed by subscriber data. Under peak events (stadium concerts, disasters, rush hour) static rules cannot react to real radio conditions, which risks congestion collapse.

2.2 Objectives

1. Model gNodeB PRB utilization from physical-layer parameters (SNR, fading, demand).
2. Build a decision engine that evaluates load every 5 seconds against policy thresholds.
3. Enforce QoS changes through a standards-inspired SBI call to a PCF.
4. Keep all thresholds, target UEs and QoS profiles in configuration, not code.
5. Provide a reproducible environment and a benchmark report.

2.3 Scope

In scope	Out of scope (for this PoC)
Statistical cell-load simulation (273 PRBs, 100 MHz, 30 kHz SCS, 3.5 GHz C-band UMa)	Real gNodeB / real 5G Core (e.g., open5GS, free5GC)
Threshold-based throttle/restore logic	ML-based or predictive control
Mock PCF with 3GPP-style REST endpoint	Real UPF rate enforcement
Docker Compose orchestration	Multi-cell or multi-slice scenarios
CSV logging and benchmark graph	Production security (auth, TLS, OAuth2 on SBI)

2.4 Success Criteria

- Engine reads load and reaches a policy decision every 5 s without crashing on partial file writes.
- API calls are issued only on state change (no redundant signalling).
- Policies are changed by editing `policies.yaml` only.
- One command (`docker compose up`) starts the full stack.
- Benchmark log and graph are generated automatically.

3. Stakeholders

Stakeholder	Interest
Arda Özgür (project owner)	Delivery, learning outcomes, portfolio evidence
Academic supervisor: Dr. Öğr. Üyesi Nergis Erdem	Technical correctness, academic rigor
IEEE 5G/6G testbed community	Reusable PoC for closed-loop QoS experimentation
Future contributors	Clear architecture, configuration and roadmap

4. Solution Architecture

Three loosely coupled services communicate over a shared Docker network:

Component	File	Responsibility
Physical-layer simulator	`sim_sionna.py`	Generates per-UE SNR (path loss + fading), computes Shannon-based spectral efficiency with a 70% practical-efficiency factor, converts UE demand (2–12 Mbps, 40–80 UEs) to required PRBs, writes `current_load.json`
Bridge / decision engine	`bridge_engine.py`	Polls load, compares to threshold, selects `normal` or `throttled` profile, calls PCF only on state change, logs every sample to `benchmark_log.csv`
Mock PCF	`mock_pcf_server.py`	Flask endpoint `POST /npcf-smpolicy/v1/policies`; validates payload, returns `201 Created`
Policy configuration	`policies.yaml`, `channel_config.json`	Thresholds, target SUPIs, 5QI/MBR profiles, channel and UE parameters
Orchestration	`docker-compose.yml`	Starts all three services with `depends_on` ordering
Analytics	`benchmark_plotter.py`	Produces the benchmark graph from the CSV log

Key design decisions

- **File-based interface between simulator and engine** keeps the PoC simple and decoupled; the engine tolerates read errors from concurrent writes.
- **State-change-only signalling** avoids flooding the core with redundant policy updates.
- **Externalised policies** let non-developers tune behaviour; VIP subscribers are protected simply by not listing them in `target_ues`.
- **Short HTTP timeout (2 s)** prevents the control loop from blocking during network stress.

5. Work Breakdown Structure

Phase	Work packages	Deliverables
1. Initiation	Literature review (O-RAN RIC, 3GPP Npcf_SMPolicyControl, 5QI); define scope and success criteria	Charter, scope statement
2. Design	Select architecture; define JSON/YAML/REST interfaces; choose parameters (273 PRB, 3.5 GHz, UMa)	Architecture diagram (Mermaid sequence diagram), config schemas

Phase	Work packages	Deliverables
3. Development	Simulator, decision engine, mock PCF, policy files	`sim_sionna.py`, `bridge_engine.py`, `mock_pcf_server.py`, `policies.yaml`
4. Integration	Containerise services; shared network; startup ordering	`docker-compose.yml`, `requirements.txt`
5. Verification	Run end-to-end; log every decision; plot results	`benchmark_log.csv`, `benchmark_report_graph.png`
6. Documentation	README, configuration guide, roadmap	`README.md`

Project window: 7 October 2026 – 19 October 2026.

6. Risk Register

#	Risk	Likelihood	Impact	Response	Outcome
R1	Simulator and engine read/write the JSON file simultaneously	Medium	Medium	Catch `JSONDecodeError`/`PermissionError`, skip the cycle	Mitigated in code
R2	PCF unreachable or slow stalls the control loop	Medium	High	2 s timeout; exception handling; engine continues	Mitigated
R3	Redundant API calls flood the core	High	Medium	Track `current_applied_policy`; signal only on change	Mitigated
R4	Missing policy file	Low	High	Built-in default protection policy	Mitigated
R5	No access to a real 5G Core in the timeframe	High	High	Mock PCF with the same endpoint contract, so a real core can be swapped in	Accepted, documented
R6	Simulated traffic unrealistic for validating control behaviour	Medium	High	Calibrate demand using `channel_config.json`	**Materialised** (see Section 7)
R7	Heavy TensorFlow/Sionna dependency slows setup	Medium	Low	Pinned official TF image in Docker	Mitigated

7. Quality Assurance and Results

7.1 Verification approach

End-to-end runs with the full stack, with every engine cycle appended to `benchmark_log.csv` (timestamp, load %, policy). `benchmark_plotter.py` renders load over time, the 80% threshold, and the samples where throttling was active.

7.2 Benchmark observations

The IEEE 5G/6G testbed run produced 720 samples at about 5 s intervals, logged between 18 October 2026 (11:59) and 20 October 2026 (12:23). Active sampling covers about 63 minutes in the first session and about 7 minutes after a roughly 47-hour pause. Observed behaviour:

- Cell load read **100%** in 718 of 720 samples (99.7%), with a mean of 99.98%.
- Only two samples dipped below 100%: 93.99% at about 946 s and 90.75% at about 3,793 s (the minimum).
- No sample fell below the 80% threshold. All 720 samples show the `throttled` policy, with no policy change after the first sample.
- The **throttled → normal recovery path was not exercised** in this run.

- The graph's time axis runs to about 174,000 s because the plotter counts from the first sample, so the 47-hour idle gap squeezes all real data against the left edge.

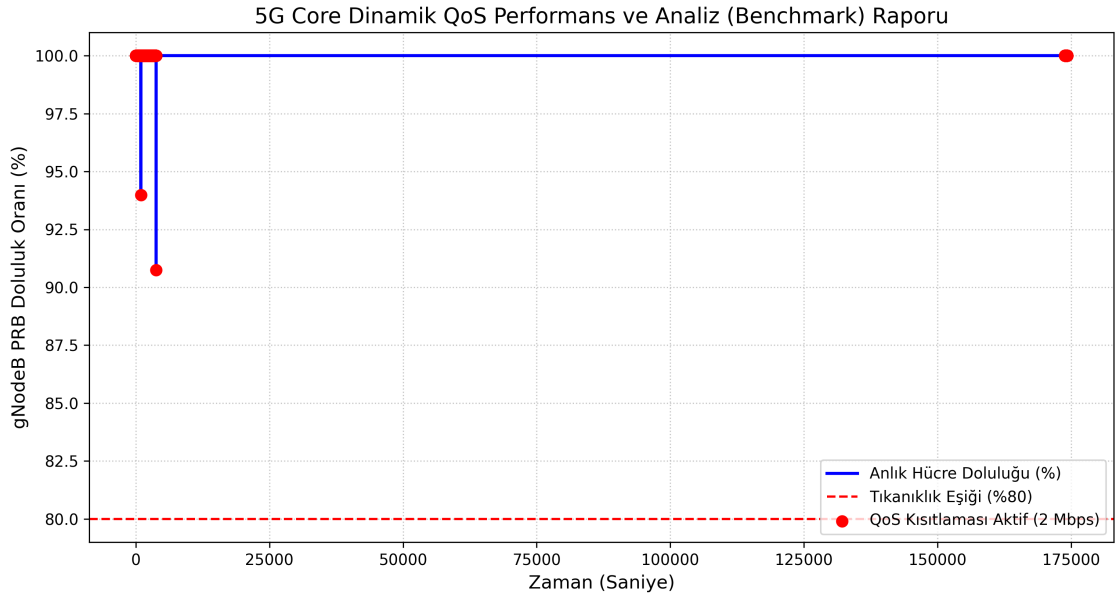


Figure 1. Cell load over time from the testbed log; red markers show samples with throttling active.

7.3 Honest assessment

The benchmark validates the *detection and enforcement* path but not the *recovery* path, and it does not demonstrate the benefit of throttling. Root causes identified from the code and configuration:

1. **Demand outpaces capacity.** 40–80 UEs at 2–12 Mbps each (average roughly 7 Mbps) implies hundreds of Mbps of demand, and low-SNR UEs need many PRBs each, so the clipped load saturates at 100%.
2. **Open loop in practice.** The throttling decision is sent to the PCF but never fed back into the simulator, so load does not respond to the control action. The loop is closed in architecture but not in the simulation.
3. **Single threshold, no hysteresis.** A load hovering near 80% could cause rapid flapping between profiles.
4. **Simulation fidelity.** The channel is a statistical model (uniform mean SNR plus Gaussian fading). The `channel_config.json` values (for example `los_probability`, `max_snr_db`) are not yet consumed by the simulator, and Sionna's full channel models are not yet used.
5. **Mocked enforcement.** The PCF prints that the policy reached the UPF; no rate limit is actually applied.

8. Lessons Learned

What went well	What I would do differently
Clean separation of simulator, engine and core made each part testable alone	Define test scenarios and expected outcomes (including a recovery case) *before* running the benchmark
Config-driven policies made tuning easy	Calibrate the simulator against target load ranges (for example 60–95%) so both states are exercised
Defensive coding kept the loop alive under errors	Wire `channel_config.json` into the simulator so config and code cannot drift apart
Docker Compose gave one-command reproducibility	Add automated tests and a CI check for the engine's decision logic

Key PM takeaway: a result that looks "successful" (the system ran and logged) is not the same as a result that meets the success criteria. Defining measurable acceptance tests up front would have exposed the saturation issue earlier.

9. Improvement Plan and Roadmap

Priority	Item	Value
High	Feed throttling effect back into the simulator (reduce target UEs' demand to the 2 Mbps cap)	Truly closed loop
High	Recalibrate demand/UE counts for a 50–100% load range; add scripted load scenarios (ramp, spike, plateau)	Exercises both transitions
High	Add hysteresis (e.g., throttle at 80%, restore at 70%)	Prevents flapping
Medium	Use the `critical_throttled` profile (already in `policies.yaml`) for loads above 95%	Multi-level control
Medium	Replace the mock PCF with open5GS or free5GC	Real-core validation
Medium	Unit tests for decision logic; CI pipeline	Quality
Low	Replace file exchange with a message bus or REST/gRPC; add TLS/OAuth2 to the SBI	Production realism
Low	ML-based load prediction for proactive throttling	Research extension

10. Methodology and Tools

- **Approach:** iterative, incremental delivery of a working vertical slice (simulate → decide → enforce → measure), then refinement.
- **Planning and tracking:** WBS, risk register, milestone-based phases.
- **Version control and docs:** Git repository with README, Mermaid diagrams, configuration guide.
- **Environment:** Docker Compose with pinned base images; `requirements.txt` for local runs.
- **Measurement:** CSV event log and automated plotting.

11. Skills Demonstrated

Project management: scoping, WBS, risk management, stakeholder analysis, quality assurance, retrospective and roadmap planning.

Technical: 5G NR concepts (PRB, SCS, 5QI, MBR, SUPI), 3GPP SBI/PCF, O-RAN closed-loop control, Python, TensorFlow, REST APIs, YAML/JSON configuration, Docker, data visualisation.

Professional: critical self-evaluation, honest reporting of limitations, documentation.

12. Appendix: Repository Map

File	Purpose
`sim_sionna.py`	Physical-layer load simulator
`channel_config.json`	Channel and UE parameters
`current_load.json`	Simulator output consumed by the engine
`bridge_engine.py`	Decision engine and PCF client
`policies.yaml`	Thresholds, target UEs, QoS profiles
`mock_pcf_server.py`	Mock 5G Core PCF (Flask)
`docker-compose.yml`	Service orchestration

File	Purpose
`requirements.txt`	Python dependencies
`benchmark_log.csv`	Per-cycle decision log
`benchmark_plotter.py`	Report generator
`benchmark_report_graph.png`, `Figure_1.png`	Benchmark graph outputs
`README.md`	Project documentation